

Lab 5 Report: Localization

Team 1

Shrika Eddula
Sammy Krem
Pyae Sone Nyo Hmine
Sydney Matthews
Jayson Lin

6.4200

April 10, 2025

1 Introduction: Shrika Eddula

Robust localization is a fundamental capability for mobile robots operating in real-world environments. In this lab, we implemented Monte Carlo Localization (MCL), a probabilistic technique that uses a particle filter to robustly estimate a robot's position and orientation within a known map. The goal of this lab was to develop, implement, and test a reliable localization system using MCL on a racecar, even in the presence of motion and sensor noise.

Our implementation consists of three key components: a particle filter that manages the overall algorithm, a motion model that predicts particle movements based on odometry data, and a sensor model that evaluates how well observed laser scans match expected measurements. We tested our system both in simulation and on a physical robot in various environments, evaluating its performance across different hallways and turns.

This report details our technical approach, including the mathematical foundation and implementation of each component, followed by an experimental evaluation that assesses the algorithm's accuracy and robustness under different conditions. We analyze the impact of various parameter settings and present quantitative results showing the system's performance across multiple trials.

2 Technical Approach: Shrika Eddula

2.1 Overview

In this project, we implemented Monte Carlo Localization (MCL) to solve the robot localization problem in a known environment with ROS2, a framework designed for robot software development. Our solution represents the robot's pose belief as a set of weighted particles that are updated through sequential motion predictions and measurement corrections. Using ROS2's publisher-subscriber model, we subscribe to racecar sensor data through callback functions, allowing for real-time updates and corrections in the localization. We publish the estimated robot's pose. This probabilistic approach, which utilizes handles uncertainty in both motion and sensing while efficiently converging to the robot's true position.

2.2 Technical Implementation

2.2.1 Particle Filter

Our particle filter implementation serves as the primary algorithm for state estimation and tracking. Each particle represents a possible robot pose as a three-dimensional vector (x, y, θ) corresponding to position and orientation in the map frame. The particle filter consists of three key components: particle initialization, a motion update, a laser scan measurement update

For initialization, we support interactive pose setting through ROS2's RViz "Pose Estimate" feature. When a pose estimate is received, we uniformly distribute particles around this position using controlled random spreads:

$$x_i = x_{init} + \mathcal{U}(-r_t, r_t) \quad (1)$$

$$y_i = y_{init} + \mathcal{U}(-r_t, r_t) \quad (2)$$

$$\theta_i = \theta_{init} + \mathcal{U}(-r_\theta, r_\theta) \quad (3)$$

where $\mathcal{U}(a, b)$ denotes a uniform random distribution with r_t and r_θ as the translational and orientational spread radius respectively. This initialization, where $r_t = 1$ and $r_\theta = \frac{\pi}{4}$ creates a cloud of potential starting positions, allowing the filter to converge even with imprecise initial estimates.

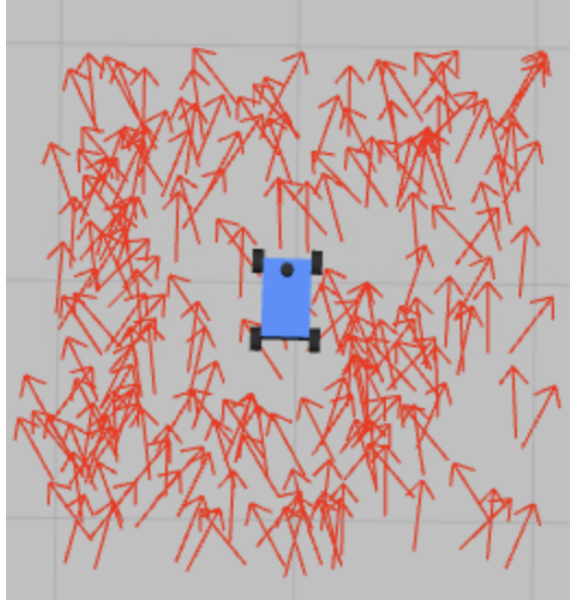


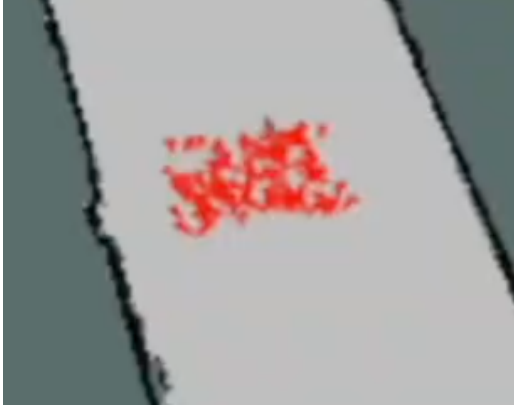
Figure 1: RViz helps verify that the implementation of the uniform spread is correct

The motion update phase processes odometry data through an odometry subscriber callback to predict particle movement. Our implementation calculates the time difference Δt between consecutive updates and extracts the linear and angular velocities from the twist component of the odometry message:

$$\Delta x = v_x \cdot \Delta t \quad (4)$$

$$\Delta y = v_y \cdot \Delta t \quad (5)$$

$$\Delta \theta = \omega_z \cdot \Delta t \quad (6)$$



((a)) The poses are uniformly initialized as shown in Figure 1.



((b)) After initialization, the poses converge closer to the robot's true pose. The longer red arrow helps show the computed average pose of all poses.

Figure 2: **RVIZ example of converging poses for localization.**

These displacements, in the robot frame, are passed to the motion model, which updates each particle position in the world frame while accounting for motion uncertainty. Afterward, the measurement update phase, triggered by a callback function subscribing to the laser scan data, evaluates how well each particle's ray-casted beams matches observed sensor data. Upon receiving laser scan data, we:

1. Down sample the laser readings by selecting 100 evenly spaced beams by angle for computational efficiency
2. Use the sensor model to calculate the likelihood of the observed range measurements based on the ray-casted beams generated from each particle's position and orientation
3. Normalize the resulting weights to form a valid probability distribution
4. Handle edge cases where all particles might receive zero weight due to poor particle distribution or sensor mismatch, by re-initializing with uniform weights

After measurement updates, we perform importance resampling to focus computational resources on the most promising regions of the state space:

$$P(\text{select particle}_i) = w_i \quad (7)$$

This stochastic resampling is implemented using `numpy.random.choice`, which effectively propagates particles with higher weights to the next iteration, while the random nature of this selection maintains diversity in the distribution.

For pose estimation, we compute a weighted average of the particle positions:

$$\hat{x} = \sum_{i=1}^N w_i \cdot x_i \quad (8)$$

$$\hat{y} = \sum_{i=1}^N w_i \cdot y_i \quad (9)$$

For orientation, we use a circular mean to properly handle the periodic nature of angles:

$$\hat{\theta} = \text{atan2} \left(\sum_{i=1}^N w_i \cdot \sin(\theta_i), \sum_{i=1}^N w_i \cdot \cos(\theta_i) \right) \quad (10)$$

This estimated pose is published as both a transform between the map frame and the particle filter frame, and as an Odometry message on the `/pf/pose/odom` topic. For the simulation, we publish the x, y, distance, and theta errors between the estimated pose and the racecar’s ground truth pose for the simulation experimental analysis below.

2.2.2 Motion Model

The motion model predicts particle movement from odometry data while incorporating motion uncertainty. A key aspect of our implementation is properly transforming odometry displacements from the robot’s local frame to the world frame for each particle:

$$\Delta x_{world} = \Delta x \cdot \cos(\theta) - \Delta y \cdot \sin(\theta) \quad (11)$$

$$\Delta y_{world} = \Delta x \cdot \sin(\theta) + \Delta y \cdot \cos(\theta) \quad (12)$$

where $(\Delta x, \Delta y)$ is the displacement in the robot’s local frame, θ is the particle’s orientation, and $(\Delta x_{world}, \Delta y_{world})$ is the resulting displacement in the world frame. This transformation accounts for the fact that the same local motion produces different global displacements depending on the robot’s orientation.

To model motion uncertainty, we add Gaussian noise to each component of the motion:

$$x_{new} = x + \Delta x_{world} + \mathcal{N}(0, \sigma_x) \quad (13)$$

$$y_{new} = y + \Delta y_{world} + \mathcal{N}(0, \sigma_y) \quad (14)$$

$$\theta_{new} = \theta + \Delta \theta + \mathcal{N}(0, \sigma_\theta) \quad (15)$$

where $\mathcal{N}(\mu, \sigma)$ represents a normal distribution with mean μ and standard deviation σ . Our noise parameters were tuned empirically to reflect the physical characteristics and dynamics of the robot:

- $\sigma_x = 0.05m$ - Standard deviation for horizontal motion noise with respect to the map frame
- $\sigma_y = 0.05m$ - Standard deviation for vertical motion noise with respect to the map frame
- $\sigma_\theta = 7$ - Standard deviation for rotational motion noise

The motion model supports a deterministic mode where noise is disabled, which is useful for testing. In this mode, particles are updated using only the transformed odometry without additional noise. Notably, the deterministic mode helped verify that the real race car interpreted odometry displacements inversely relative to the simulation, so we flipped the signs of odometry displacement for the real race car.

2.2.3 Sensor Model

Our sensor model evaluates the likelihood of observed laser scan data given a particle’s position and the known map. This is implemented as a beam-based model that considers the probability distribution of range measurements.

For computational efficiency, we precompute a lookup table that represents the likelihood $P(z|d)$ of measuring a range z given an expected range d . This table combines four probability distributions that model different aspects of LIDAR behavior:

$$P(z|d) = \alpha_{hit} \cdot P_{hit}(z|d) + \alpha_{short} \cdot P_{short}(z|d) + \alpha_{max} \cdot P_{max}(z|d) + \alpha_{rand} \cdot P_{rand}(z|d) \quad (16)$$

where:

1. **Hit Model** (P_{hit}): A Gaussian distribution centered on the expected range, modeling correct measurements with sensor noise:

$$P_{hit}(z|d) = \frac{1}{\sqrt{2\pi\sigma_{hit}^2}} \exp\left(-\frac{(z-d)^2}{2\sigma_{hit}^2}\right) \quad (17)$$

2. **Short Model** (P_{short}): A linear model representing unexpected obstacles causing shorter readings:

$$P_{short}(z|d) = \begin{cases} \frac{2}{d} \left(1 - \frac{z}{d}\right) & \text{if } 0 \leq z \leq d \text{ and } d \neq 0 \\ 0 & \text{otherwise} \end{cases} \quad (18)$$

3. **Max Model** (P_{max}): A spike distribution at the maximum range, modeling cases where the beam doesn't hit any obstacle:

$$P_{max}(z|d) = \begin{cases} 1 & \text{if } z = z_{max} \\ 0 & \text{otherwise} \end{cases} \quad (19)$$

4. **Random Model** (P_{rand}): A uniform distribution representing random noise or unexplained measurements:

$$P_{rand}(z|d) = \frac{1}{z_{max}} \quad (20)$$

The model parameters were tuned to optimize performance in our testing environment:

- $\alpha_{hit} = 0.74$ - Weight for the hit model
- $\alpha_{short} = 0.07$ - Weight for the short model
- $\alpha_{max} = 0.07$ - Weight for the max model
- $\alpha_{rand} = 0.12$ - Weight for the random model
- $\sigma_{hit} = 8.0$ - Standard deviation for the hit model

During runtime, the sensor model simulates expected laser readings for each particle using ray-casting from the PyScanSimulator2D library. Both the simulated and observed readings are converted to the map's pixel scale to align with the precomputed table indices.

The likelihood of each observation given the expected reading is then looked up in the precomputed table. To compute the overall particle weight, we combine the likelihoods of all beams using a log-space computation to prevent numerical underflow:

$$w_i = \exp\left(\sum_{j=1}^B \log(P(z_j|d_{i,j}) + \epsilon)\right) \quad (21)$$

where B is the number of beams, z_j is the j -th observed range, $d_{i,j}$ is the j -th expected range for particle i , and ϵ is a small constant (1e-12 in our implementation) to avoid numerical issues when taking logarithms of potentially zero probabilities.

2.3 Design Considerations

Our implementation balances several key design considerations:

Efficiency and Accuracy Tradeoffs We optimized the algorithm by using 200 particles, down sampling laser scans, and pre-computing the sensor model table. These choices maintain acceptable accuracy while ensuring the algorithm runs in real-time on the robot hardware.

Robustness We incorporated several mechanisms for enhanced robustness, including proper thread synchronization, numerical stability safeguards, and handling of edge cases like zero-weight particles. The use of circular statistics for orientation calculations prevents discontinuities in angular representations.

Parameter Tuning The system performance relies on carefully tuned parameters across the motion model, sensor model, and particle filter. We experimentally determined these values to achieve reliable localization across different environments and motion patterns.

3 Experimental Evaluation: Sammy Krem, Jayson Lin, Sydney Matthews, Pyae Sone Nyo Hmine

3.1 MCL Algorithm on Different Hallways and Turns

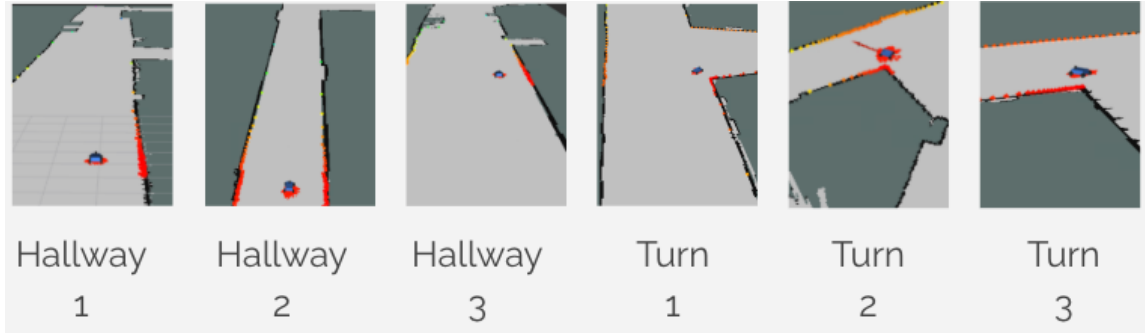


Figure 3: These are the hallways and turns used for our experiments on the racecar. They are referenced in the experimental tables below.

We tested our localization algorithm in simulations and real experiments across three hallways and three turns to evaluate performance in unique and generic environments. We used the wall follower for simulation and teleoperation for the actual robot to operate these maneuvers. Success was defined as particles aligning with the robot at the end of its path, allowing for some drift as long as it was corrected by the end. The robot performed well in Hallways 1 and 3, which had distinct features like doorways and vending machines, but struggled in Hallway 2 due to its lack of unique characteristics. Turn experiments performed better, as the scans provided more distinct features for localization.

3.2 Initial Noise Parameters

Our initial parameter set performed poorly in simulation. Table I shows high translational errors (up to 1.5 meters).

Table I: Hallway Error Across 10 Trials (Std Dev: $X = 0.1$ m, $Y = 0.02$ m, $\theta = 12^\circ$)

	X error (m)	Y error (m)	Distance error (m)	$ \theta$ error (deg)
Hallway 1	0.53	0.030	0.549	1.15°
Hallway 2	1.534	0.0461	1.543	0.97°
Hallway 3	0.0671	0.269	0.296	1.68°
Average	0.712	0.116	0.795	1.27°



Figure 4: **Hallway 2 had no successes and the highest translational error**, primarily due to localization drift, seen by the separation between the particles and the race car

Table II: Unsatisfactory Hallway Success rate

	Success % (10 trials)
Hallway 1	0.5
Hallway 2	0.0
Hallway 3	0.7
Average	0.4

Table I showcases the hallway’s significant average error in meters. Our success rates averaged 40% in 10 trials, with Hallway 2 experiencing drift every trial, highlighting the need to tune parameters

3.3 Increased Noise Analysis

Increasing y and θ noise improved success rates by countering more drift and expanding the distribution of potential states. However, it also raised error, especially in Hallway 2, and while drift was reduced, some occasional drift still occurred due to over-exploration.

Table III: Hallway Error Across 10 Trials (Std Dev: $X = 0.1$ m, $Y = 0.1$ m, $\theta = 45^\circ$)

	X error (m)	Y error (m)	Distance error (m)	$ \theta$ error (deg)
Hallway 1	0.480	0.051	0.491	1.15°
Hallway 2	2.204	0.0514	2.211	1.61°
Hallway 3	0.101	0.546	0.555	3.49°
Average	0.597	0.212	0.753	2.42°

Table IV: Improved Hallway Success (Std Dev: $X = 0.1$ m, $Y = 0.1$ m, $\theta = 45^\circ$)

	Success % (10 trials)
Hallway 1	0.8
Hallway 2	0.5
Hallway 3	0.9
Average	0.73

3.4 Best parameters for Strong Localization Performance

We iterated through various parameter combinations to find the best values, measuring the success rate at each step. The parameter set of (0.05, 0.05, 45) emerged as the best.

Table V: Strong Hallway Error (10 Trials)

	X error (m)	Y error (m)	Distance error (m)	\theta error (deg)
Hallway 1	0.318	0.035	0.326	1.20°
Hallway 2	0.589	0.043	0.592	1.71°
Hallway 3	0.065	0.323	0.329	2.79°
Average	0.324	0.134	0.415	1.90°

Table VI: Strong Hallway Success

	Success % (10 trials)
Hallway 1	0.9
Hallway 2	0.8
Hallway 3	1.0
Average	0.9

Tables V and VI show improved error and success rate. This set of standard deviations allowed the particles to explore enough of the state space without straying too far from the true pose, balancing between exploration and accuracy for stable localization.

Table VII: Low Turn Errors (10 Trials)

	X error (m)	Y error (m)	Distance error (m)	\theta error (deg)
Turn 1	0.269	0.175	0.359	7.44°
Turn 2	0.154	0.2807	0.35	2.69°
Turn 3	0.184	0.165	0.281	3.45°
Average	0.202	0.207	0.33	4.52°

Table VIII: Strong Turn Success (10 Trials)

	Success % (10 trials)
Turn 1	1.0
Turn 2	1.0
Turn 3	1.0
Average	1.0

Tables VII and VIII show that turning was 100% successful and precise, with minimal translational errors and no drift, due to the distinct geometry of turns allowing better localization. However, theta errors were larger as turns caused greater orientation changes.

3.5 Sim-to-Real Integration

Transitioning from simulation to real-life experiments, we tuned our noise parameters to fit robot performance. We re-ran our hallway and turn trials and tabulated results below. The major parameter change most effective for the real robot was a drastic reduction in theta noise, from 45° to 20° .

Table IX: Hallway Success Across 5 Trials
Old Std Dev: $X = 0.05$ m, $Y = 0.05$ m, $\theta = 45^\circ$

	Success % (5 trials)
Hallway 1	0.4
Hallway 2	0.0
Hallway 3	0.6
Average	0.33

Table X: Hallway Success Across 5 Trials
New Std Dev: $X = 0.05$ m, $Y = 0.05$ m, $\theta = 20^\circ$

	Success % (5 trials)
Hallway 1	0.6
Hallway 2	0.4*
Hallway 3	0.8
Average	0.53

Table XI: Turn Success Across 5 Trials ($\theta = 20^\circ$)

	Success % (5 trials)
Turn 1	0.8
Turn 2	0.8
Turn 3	0.6
Average	0.73

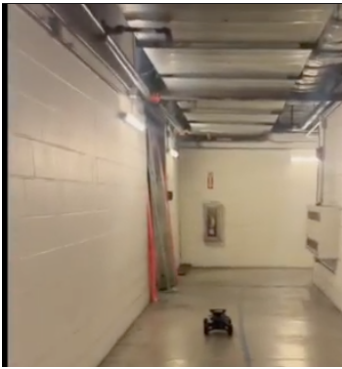
As seen in Table IX, our hallway trials, particularly in Hallway 2, exhibited some localization inaccuracies. To address this, we reduced the theta standard deviation from 45° (used in simulation) to smaller values, such as 20° , which significantly improved the success rate of our particle filter as seen in Table X.

In simulation, higher theta noise was effective because the laser scans perfectly matched the known map, allowing the robot to rely more on the map for localization rather than on odometry. This prevented sensor drift and helped achieve more accurate localization. Since both the map and laser scans were consistent and accurate, the robot could tolerate higher noise in its rotational estimates (theta) without negatively impacting localization. The system was confident in using the map, as it

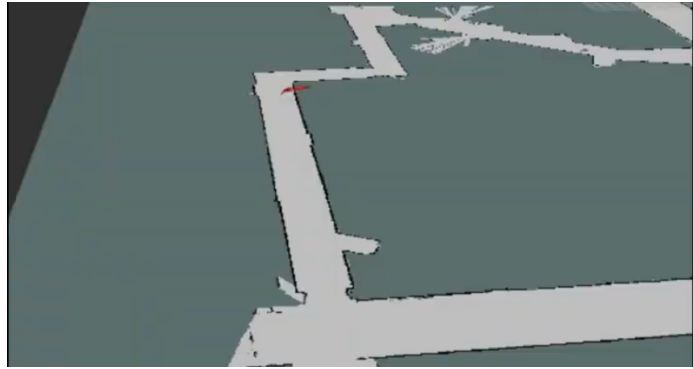
accurately represented the environment, and small variations in orientation could be compensated for.



Figure 5: **Foreign objects obstruct LIDAR scans, affecting the robot’s localization.** In the figure, the real racecar is performing Turn 3, but the presence of people does not align with the robot’s map. This causes the initially correct pose on the left to shift to an incorrect orientation on the right..



((a)) Observe the translucent panes on the left at the end of the hallway.



((b)) Due to the provided map’s inability to represent the translucent panes, the robot’s localization is quite off when attempting to make a right turn here at Hallway 2.

Figure 6: **Map inaccuracies lead to localization errors**

However, in real-world experiments, laser scans didn’t always match the map due to environmental variations like other robots, people, and new furniture, making reliance on the map prone to drift. For example, as seen in Tables 7 and 8, Hallway 2 was only accurately navigated 2 times with reduced theta noise of 20° . The reduced theta helped mitigate drift when the map couldn’t represent features like the translucent panes at the hallway’s end.

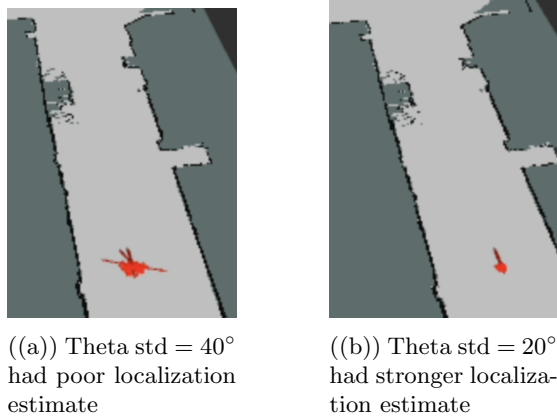


Figure 7: **Smaller theta standard deviation resulted in improved localization precision and accuracy.**

By making the theta component of the motion model more deterministic, the robot relied more on odometry and less on imperfect map data. We found that higher speeds were necessary for Hallway 2, as traversing longer distances increased the variation in laser scans, making each scan more distinct and improving the filter’s ability to localize by matching sensor data to more accurate parts of the map. This increased scan variation helped the filter better determine the robot’s position. This is counterintuitive because higher speeds usually lead to increased sensor noise and odometry errors, but in this case, the increased scan variation actually enhanced localization. As seen in Table IX, turns without obstacles like people or robots performed better due to clearer navigation paths, leading to more accurate data.

4 Conclusion: Sydney Matthews

The localization lab demonstrated the application of fundamental robotics concepts to real-world scenarios, bridging simulation and hardware. By integrating a motion model, sensor model, and particle filter, we created a robust Monte Carlo Localization algorithm successful in multiple unique environments.

Our team proved the accuracy of our model in simulation for both unique and generic areas of the map. Repeated trials and parameter tuning enabled the transition to hardware application. Extensive testing and data accumulation, both visual and from a ROS graphical interface, helped us further optimize the model.

Future work could determine the cause of the theta standard deviation discrepancy between simulation and real robot. Additionally, our model could be optimized to decrease errors in common dead reckoning locations and improve performance in generic environments.

5 Lessons Learned

5.1 Pyae Sone Nyo Hmine

I find it greatly useful to have a great overall understanding of the lab, including all the components, in order to effectively integrate, debug, and collaborate together with my teammates.

5.2 Shrika Eddula

The transition from simulation to physical hardware reinforced the challenges of real-world deployment for me, where sensor noise, computational constraints, and environmental variations require careful system calibration. I also learned the importance of parameter tuning in robust localization systems, particularly how sensor model configuration and motion noise parameters significantly impact the algorithm's performance in different environments. An important collaboration aspect I learned was starting work very early and working in parallel.

5.3 Sammy Krem

I learned how useful ROS2 can for debugging purposes. For example, when we thought our particles weren't resampling and we had looked through our code rigorously, we realized that it was our broken LIDAR causing this issue. Related to this, I also learned the importance of connecting concepts we learned in lecture with the technical implementation, as this helped us debug code and make the particle filter simulation work. This lab helped me improve my patience when it comes to dealing with issues, the power of dividing and conquering work, and how useful collaborating and bouncing ideas can be when coding algorithms.

5.4 Sydney Matthews

I learned how powerful a tool simulation can be. When conducting our experimental evaluation, it was very beneficial to compare our real-life trials to our simulated data. Also, being able to test our model first in sim to confirm that it's working before moving to the real car saved us a lot of time debugging. Our team also developed a better plan for collaborating for this lab. I learned the how useful dividing up the work is and having a meeting plan helped us stay on track.

5.5 Jayson Lin

The transition from simulation to physical hardware has challenges beyond parameter tuning. In this lab, the real racecar interpreted odom displacements in the opposite direction, and we also ran into issues with the LIDAR connection such that a restart of the racecar was the solution instead of debugging the code. These experiences reinforce the importance of carefully tracking and trying all considerations in small incremental steps when working with hardware. Had we not taken a step back and considered each issue incrementally, we might have been stuck in an endless mess of undiagnosable errors stacked atop each other. Thankfully, through the collaborative effort of my teammates we were able to quickly explore possible problem causes and come to a timely solution.